

```
typedef std::string AddressLines[4];           //每个人的地址有 4 行,  
                                              //每行是一个 string
```

由于 AddressLines 是个数组, 如果这样使用 new:

```
std::string* pal = new AddressLines; //注意, "new AddressLines" 返回  
                                   //一个 string*, 就像  
                                   //"new string[4]" 一样。
```

那就必须匹配“数组形式”的 delete:

```
delete pal;           //行为未有定义!  
delete [ ] pal;       //很好。
```

为避免诸如此类的错误, 最好尽量不要对数组形式做 typedefs 动作。这很容易达成, 因为 C++ 标准程序库 (条款 54) 含有 string, vector 等 templates, 可将数组的需求降至几乎为零。例如你可以将本例的 AddressLines 定义为“由 strings 组成的一个 vector”, 也就是其类型为 vector<string>。

请记住

- 如果你在 new 表达式中使用 [], 必须在相应的 delete 表达式中也使用 []。如果你在 new 表达式中不使用 [], 一定不要在相应的 delete 表达式中使用 []。

条款 17: 以独立语句将 newed 对象置入智能指针

Store newed objects in smart pointers in standalone statements.

假设我们有个函数用来揭示处理程序的优先权, 另一个函数用来在某动态分配所得的 Widget 上进行某些带有优先权的处理:

```
int priority();  
void processWidget(std::tr1::shared_ptr<Widget> pw, int priority);
```

由于谨记“以对象管理资源”(条款 13) 的智慧铭言, processWidget 决定对其动态分配得来的 Widget 运用智能指针 (这里采用 tr1::shared_ptr)。

现在考虑调用 processWidget:

```
processWidget(new Widget, priority());
```

等等，不要考虑这个调用形式。它不能通过编译。`tr1::shared_ptr` 构造函数需要一个原始指针（raw pointer），但该构造函数是个 `explicit` 构造函数，无法进行隐式转换，将得自 `"new Widget"` 的原始指针转换为 `processWidget` 所要求的 `tr1::shared_ptr`。如果写成这样就可以通过编译：

```
processWidget(std::tr1::shared_ptr<Widget>(new Widget), priority());
```

令人惊讶的是，虽然我们在此使用“对象管理式资源”（object-managing resources），上述调用却可能泄漏资源。稍后我再详加解释。

编译器产出一个 `processWidget` 调用码之前，必须首先核算即将被传递的各个实参。上述第二实参只是一个单纯的对 `priority` 函数的调用，但第一实参 `std::tr1::shared_ptr<Widget>(new Widget)` 由两部分组成：

- 执行 `"new Widget"` 表达式
- 调用 `tr1::shared_ptr` 构造函数

于是在调用 `processWidget` 之前，编译器必须创建代码，做以下三件事：

- 调用 `priority`
- 执行 `"new Widget"`
- 调用 `tr1::shared_ptr` 构造函数

C++ 编译器以什么样的次序完成这些事情呢？弹性很大。这和其他语言如 Java 和 C# 不同，那两种语言总是以特定次序完成函数参数的核算。可以确定的是 `"new Widget"` 一定执行于 `tr1::shared_ptr` 构造函数被调用之前，因为这个表达式的结果还要被传递作为 `tr1::shared_ptr` 构造函数的一个实参，但对 `priority` 的调用则可以排在第一或第二或第三执行。如果编译器选择以第二顺位执行它（说不定可因此生成更高效的代码，谁知道！），最终获得这样的操作序列：

1. 执行 `"new Widget"`
2. 调用 `priority`
3. 调用 `tr1::shared_ptr` 构造函数

现在请你想想，万一对 `priority` 的调用导致异常，会发生什么事？在此情况下 `"new Widget"` 返回的指针将会遗失，因为它尚未被置入 `tr1::shared_ptr` 内，后者是我们期盼用来防卫资源泄漏的武器。是的，在对 `processWidget` 的调用过程中可能引发资源泄漏，因为在“资源被创建（经由 `"new Widget"`）”和“资源被

转换为资源管理对象”两个时间点之间有可能发生异常干扰。

避免这类问题的办法很简单: 使用分离语句, 分别写出 (1) 创建 Widge, (2) 将它置入一个智能指针内, 然后再把那个智能指针传给 processWidget:

```
std::tr1::shared_ptr<Widget> pw(new Widget);    //在单独语句内以
                                                // 智能指针存储
                                                // newed 所得对象。
processWidget(pw, priority());                  //这个调用动作绝不至于造成泄漏。
```

以上之所以行得通, 因为编译器对于“跨越语句的各项操作”没有重新排列的自由 (只有在语句内它才拥有那个自由度)。在上述修订后的代码内, "newWidget" 表达式以及“对 tr1::shared_ptr 构造函数的调用”这两个动作, 和“对 priority 的调用”是分隔开来的, 位于不同语句内, 所以编译器不得在它们之间任意选择执行次序。

请记住

- 以独立语句将 newed 对象存储于 (置入) 智能指针内。如果不这样做, 一旦异常被抛出, 有可能导致难以察觉的资源泄漏。

4

设计与声明

Designs and Declarations

所谓软件设计，是“令软件做出你希望它做的事情”的步骤和做法，通常以颇为一般性的构想开始，最终演变成十足的细节，以允许特殊接口（interfaces）的开发。这些接口而后必须转换为 C++ 声明式。本章中我将对良好 C++ 接口的设计和声明发起攻势。我以或许最重要、适合任何接口设计的一个准则作为开端：“让接口容易被正确使用，不易被误用”。这个准则设立了一个舞台，让其他更专精的准则对付一大范围的题目，包括正确性、高效性、封装性、维护性、延展性，以及协议的一致性。

以下准备的材料并不覆盖你需要知道的优良接口设计的每一件事，但它强调某些最重要的考虑，对某些最频繁出现的错误提出警告，为 class、function 和 template 设计者经常遭遇的问题提供解答。

条款 18：让接口容易被正确使用，不易被误用

Make interfaces easy to use correctly and hard to use incorrectly.

C++ 在接口之海漂浮。function 接口、class 接口、template 接口……每一种接口都是客户与你的代码互动的手段。假设你面对的是一群“讲道理的人”，那些客户企图把事情做好。他们想要正确使用你的接口。这种情况下如果他们对任何一个接口的用法不正确，你至少也得负一部分责任。理想上，如果客户企图使用某个接口却没有获得他所预期的行为，这个代码不该通过编译；如果代码通过了编译，它的作为就该是客户所想要的。

欲开发一个“容易被正确使用，不易被误用”的接口，首先必须考虑客户可能做出什么样的错误。假设你为一个用来表现日期的 class 设计构造函数：